Outside contributions to the Agentic Engineering Framework: draft plan

2026-11-02

Outside contributors will send pull requests on GitHub against `bleeding-edge`; the maintainer alone merges, after tests and an independent review. This draft answers Mehdi's proposal of 2026-11-02 and is shared for review before the maintainer signs it off.

Short answer to your question ("Does this fit how you want to work? Anything you would change before I open PR 1?"): yes, it fits, with the changes listed under "Your proposal, point by point". Please wait with PR 1 until the maintainer has signed this plan off; PR 1 then carries the corrections listed under "Your first two pull requests".

How a contribution flows

Every change starts as a GitHub issue and lands through one pull request that only the maintainer merges.

1. **Issue first.** Open or pick an issue and agree the approach in it before writing code. The maintainer's intake turns each issue into a framework task (for example #104 became T-4935).
2. **Fork and branch.** Fork the repository on GitHub and branch from `bleeding-edge` in your fork.
3. **Commit.** Each commit names the intake task and the issue, e.g. `T-4935 (#104): ...`, and carries a `Signed-off-by:` line (`git commit -s`).
4. **Open the pull request** against `bleeding-edge`, linking its issue ("Fixes #106"). One pull request per issue.
5. **Automatic checks** run on GitHub's hosted runners. For a first-time contributor the maintainer approves the run with one click.
6. **Review and deeper tests** by the framework's agents (next sections). The result is a one-screen recommendation on the pull request: merge, change or refuse.
7. **The maintainer merges** into `bleeding-edge`. Nobody else can.
8. **Field tests.** The change ships in the next pre-release tag (`be-tag`), which the maintainer's own projects install and test.

9. **Release.** `master` moves forward only at a release, once those field tests are fine.

What every pull request must bring

The contributor delivers the test and the evidence; a pull request without them is refused with a one-line reason.

- ☐ A linked issue and its intake task (`T-XXXX (#NNN)` in each commit)
- ☐ `Signed-off-by:` on every commit (Developer Certificate of Origin)
- ☐ The approach, and the class of defect it closes, not only the instance
- ☐ A test that fails before the change and passes after it
- ☐ Evidence: the commands run and their output
- ☐ `fw audit` and the tests touched, with their results
- ☐ `fw doctor`: green, or the components absent by choice
- ☐ What was not tested

Contributions are licensed under Apache-2.0, like the project (section 5 of the licence); there is no contributor licence agreement.

What happens on our side

Until the maintainer merges, a pull request's code runs only on throwaway machines, never on the maintainer's own hosts.

Stage	What runs	Where
Automatic checks	Tests, structure audit, lint and the framework's task checks, all through framework commands	GitHub-hosted runners (free for public repositories)
Read review	Independent review agents read the change and the linked task; they do not run it	Isolated review seats
Deeper tests	The fuller test suite against the change	The framework's own isolated test hosts (planned, high priority)
Recommendation	One screen on the pull request: what it changes, its risk level, test results,	Posted as a required check

Stage	What runs	Where
	merge / change / refuse	
Merge	The maintainer presses Merge into `bleeding-edge`; GitHub requires the checks and the review check to be green first	GitHub
Field tests	The maintainer's own projects install the next `be-tag` and report works or fails	After the merge

A change from outside starts one risk level higher than the same change made internally, so its review is one step stricter. Documentation-only pull requests skip the deeper tests.

Release channels: bleeding edge, pre-release, stable

A change moves through three channels, each with its own rhythm and proof.

Channel	What gets in	Rhythm	Proof needed
Bleeding edge (`bleeding-edge` branch, `be-tags`)	Everything new, including merged pull requests	Daily	Basic tests green
Pre-release	Everything from bleeding edge shown to work by 2 to 3 different agents in the field	Weekly	Field-test verdicts
Stable (`master`, release tags)	Everything that ran as pre-release for at least 2 weeks	Monthly	Used by at least two different projects in those 2 weeks, and no open failure against it

An urgent security fix may skip the waiting: field-proven by two agents, then straight to pre-release and stable, with the maintainer's explicit go. Pull requests

always target `bleeding-edge`; check your change against the latest pre-release, as you proposed.

Changes to the repository

GitHub becomes the source repository, so a merged pull request can no longer be lost to a mirror sync, as happened to #76, #77 and #79.

- **Source of truth:** today the maintainer's private git server pushes to GitHub. That reverses: work lands on GitHub, and the private server becomes its mirror.
- **Branch protection:** `bleeding-edge` and `master` refuse force pushes and deletion. Merging into `bleeding-edge` requires green checks and the review check.
- **CI:** `test.yml` also runs for pull requests to `bleeding-edge` (today it runs only for `main` and `master`). The workflow calls framework commands only, so the same checks can later move to self-hosted isolated runners.
- **Templates and documents:** a pull request template with the checklist above; `CONTRIBUTING.md` corrected (there is no `main` branch); a `SECURITY.md`.
- **Security reports** go through GitHub's private vulnerability reporting, never a public issue.
- **Accounts:** contributors use their own GitHub accounts and need no rights on the repository; a pull request from a fork can only propose.

The pilot and its limits

Mehdi is the first contributor, as a pilot; the process opens wider only after it has held up end to end.

- **Queue:** at most 3 outside pull requests wait for the maintainer at once; further ones are labelled "waiting" until a slot frees.
- **High-risk changes** (gates, hooks, release machinery) need a three-vendor review panel that cannot run yet; such pull requests wait until it can.
- **Pull requests with code** wait for the isolated test hosts to exist. Documentation-only ones can go ahead.
- **Refusals** are one line on the pull request, drafted by the framework's agent and approved by the maintainer.

A reasonable first pull request, as Mehdi proposed: no code, only the documents and settings in the previous section.

Your proposal, point by point

Nothing in Mehdi's proposal was dropped; four points were kept and four changed, mostly made stricter.

#	Your point	Outcome	Note
1	Fork, branch from and target `bleeding-edge`; `fix/` and `feat/` branch names; `master` only at release	Kept	Where a branch starts and how it is named are guidance, not checked
2	One PR per issue; "Fixes #N"; docs exempt; approach agreed first	Changed	One PR per issue kept; the docs-only exemption is still open
3	Task reference `T-4935 (#104): ...`	Kept	The intake task must exist on `bleeding-edge` before the PR
4	Pull request template	Kept	It carries the evidence listed above
5	CI on `bleeding-edge`, first-run approval	Changed	Plus the framework's own checks, through framework commands, and deeper tests on isolated hosts
6	Maintainer approves and merges; agents review before the release	Changed	Review and tests now happen before the merge, at one risk level higher; a queue of 3
7	Protect `bleeding-`	Changed	Extended to

#	Your point	Outcome	Note
	`edge` against force push		`master` and to deletion
8	Apache-2.0 section 5, no CLA	Changed	Kept, plus a DCO sign-off on every commit

Your first two pull requests, and the step after

Your start order stands.

1. **PR 1, no code:**
   - `CONTRIBUTING.md` rewritten from this plan. The repository has no `main` branch: it says `master` for releases and `bleeding-edge` for everything else; fork, branch from `bleeding-edge`, open the PR against it.
   - The pull request template with the checklist above.
   - `bleeding-edge` added to the branches in `test.yml`.
2. **PR 2, a small fix to run the whole circuit once:** for example #106 (the empty `fw upgrade` preview), checked on the latest pre-release first, as you proposed.
3. **After the pilot: the CI check from our 28 October conversation.** On every pull request, re-verify the linked task, its acceptance criteria, the human ticks and its Verification commands. Today the framework's task checks cover part of this; the full check becomes its own step once the circuit has worked end to end.

Where this work is tracked

The work sits in one arc, **github-contributions** (arc-027): "an outside contributor opens a pull request against `bleeding-edge` on GitHub, sees CI run on it, the maintainer merges it, and the change ships in the next pre-release tag without being erased by the mirror sync".

The design runs through a fixed chain of roles, one task per step; each step needs an independent review and, where it decides something, the maintainer's approval before the next starts.

Task	What it is	State
T-5006	Inception: outside	Open; decided at the end

Task	What it is	State
	contributions through GitHub pull requests (your proposal)	of the chain
T-5007	Inception: the maintainer's own projects report bugs and requests as GitHub issues	Open; internal, not part of this review
T-5021	Step 1: requirements	Done, two review rounds; awaiting the maintainer's sign-off
T-5022	Step 2: threat model	Next
T-5023	Step 3: security floor and phasing	Waiting
T-5024	Step 4: architecture	Waiting
T-5025	Step 5: evidence (how each requirement will be proven)	Waiting
T-5026	Step 6: decision logic in pseudo-code	Waiting
T-5027	Step 7: independent review panel	Waiting
T-5028	Step 8: plan of build tasks	Waiting
T-5041	Release channels: bleeding edge, pre-release, stable	Model decided; to be built
T-5009	The framework's own isolated test hosts	High priority, next
T-5008	Review seats isolated from the host's processes	Partly shipped (`v1.8.16-be.26`)
T-4997	The third review vendor's seat, needed for high-risk pull requests	Open
T-5030	Possible move of the	Deferred

Task	What it is	State
	repository into a GitHub organisation	

Open questions

Four points are still open; your view on each would help before the maintainer signs the plan off.

- ☐ **Documentation-only pull requests:** may they skip the linked-issue rule, as you proposed?
- ☐ **Branch names:** should `fix/<issue>-<keyword>` and `feat/<issue>-<keyword>` become a rule, or stay guidance?
- ☐ **Template content:** anything you would add to or drop from the checklist above?
- ☐ **End of the pilot:** what would show you that the process works well enough to open it to others?

Not for this review: how the framework relays bug reports from the maintainer's own projects to GitHub issues. That part is internal and decided separately.